

# PYTHON DICTIONARY

# CREATE A DICTIONARY

Python dictionary is a collection of items, similar to lists and tuples. However, unlike lists and tuples, each item in a dictionary is a key-value pair (consisting of a key and a value).

We create a dictionary by placing key: value pairs inside curly brackets {}, separated by commas. For example,

# creating a dictionary

```
country_capitals = {  
    "Germany": "Berlin",  
    "Canada": "Ottawa",  
    "England": "London"  
}
```

# printing the dictionary

```
print(country_capitals)
```

Output

```
{'Germany': 'Berlin', 'Canada': 'Ottawa', 'England': 'London'}
```



## Notes:

Dictionary keys must be immutable, such as tuples, strings, integers, etc. We cannot use mutable (changeable) objects such as lists as keys.

We can also create a dictionary using a Python built-in function dict().

Using the dict() method to make a dictionary:

```
thisdict = dict(name = "John", age = 36, country = "Norway")  
print(thisdict)
```

We can access the value of a dictionary item by placing the key inside square brackets.

```
country_capitals = {  
    "Germany": "Berlin",  
    "Canada": "Ottawa",  
    "England": "London"  
}
```

# access the value of keys

```
print(country_capitals["Germany"])    # Output: Berlin  
print(country_capitals["England"])    # Output: London
```

# Python Dictionary get()

```
scores = {  
    'Physics': 67,  
    'Maths': 87,  
    'History': 75  
}
```

```
result = scores.get('Physics')
```

```
print(result)  # 67
```

get() method returns:

the value for the specified key if key is in the dictionary.

None if the key is not found and value is not specified.

value if the key is not found and value is specified.



# How does get() work for dictionaries?

```
person = {'name': 'Phill', 'age': 22}  
print('Name: ', person.get('name'))
```

```
print('Age: ', person.get('age'))
```

# value is not provided

```
print('Salary: ', person.get('salary'))
```

# value is provided

```
print('Salary: ', person.get('salary', 0.0))
```

Output

Name: Phill

Age: 22

Salary: None

Salary: 0.0



**The keys of a dictionary must be unique. If there are duplicate keys, the later value of the key overwrites the previous value.**

```
hogwarts_houses = {  
    "Harry Potter": "Gryffindor",  
    "Hermione Granger": "Gryffindor",  
    "Ron Weasley": "Gryffindor",  
    # duplicate key with a different house  
    "Harry Potter": "Slytherin"  
}  
print(hogwarts_houses)
```

### **Output**

```
{'Harry Potter': 'Slytherin', 'Hermione Granger': 'Gryffindor', 'Ron Weasley': 'Gryffindor'}
```

Here, the key Harry Potter is first assigned to Gryffindor. However, there is a second entry where Harry Potter is assigned to Slytherin.

As duplicate keys are not allowed in a dictionary, the last entry Slytherin overwrites the previous value Gryffindor.



We can add an item to a dictionary by assigning a value to a new key.

```
country_capitals = {  
    "Germany": "Berlin",  
    "Canada": "Ottawa",  
}  
# add an item with "Italy" as key and "Rome" as its value  
country_capitals["Italy"] = "Rome"  
print(country_capitals)
```

We can use the del statement to remove an element from a dictionary.

```
country_capitals = {  
    "Germany": "Berlin",  
    "Canada": "Ottawa",  
}  
# delete item having "Germany" key  
del country_capitals["Germany"]  
print(country_capitals)
```

The pop() method removes and returns an element from a dictionary having the given key.

# create a dictionary

```
marks = { 'Physics': 67, 'Chemistry': 72, 'Math': 89 }
```

```
element = marks.pop('Chemistry')
```

```
print('Popped Marks:', element)
```

# Output: Popped Marks: 72

If we need to remove all items from a dictionary at once, we can use the clear() method.

```
country_capitals = {
```

```
    "Germany": "Berlin",
```

```
    "Canada": "Ottawa",
```

```
}
```

```
# clear the dictionary
```

```
country_capitals.clear()
```

```
print(country_capitals)
```



Python dictionaries are mutable (changeable). We can change the value of a dictionary element by referring to its key. For example,

```
country_capitals = {  
    "Germany": "Berlin",  
    "Italy": "Naples",  
    "England": "London"  
}
```

```
# change the value of "Italy" key to "Rome"  
country_capitals["Italy"] = "Rome"
```

```
print(country_capitals)
```

### **Output**

```
{'Germany': 'Berlin', 'Italy': 'Rome', 'England': 'London'}
```



**The update() method updates the dictionary with the elements from another dictionary object or from an iterable of key/value pairs.**

```
marks = {'Physics':67, 'Maths':87}
internal_marks = {'Practical':48}
marks.update(internal_marks)
print(marks)
# Output: {'Physics': 67, 'Maths': 87, 'Practical': 48}
```

```
d = {1: "one", 2: "three"}
d1 = {2: "two"}
# updates the value of key 2
d.update(d1)
print(d)
```

**Output**

```
{1: 'one', 2: 'two'}
```

**Note:** The update() method adds element(s) to the dictionary if the key is not in the dictionary. If the key is in the dictionary, it updates the key with the new value.



## update() When Tuple is Passed

```
dictionary = {'x': 2}
dictionary.update([('y', 3), ('z', 0)])
print(dictionary)
```

### Output

```
{'x': 2, 'y': 3, 'z': 0}
```

Here, we have passed a list of tuples [('y', 3), ('z', 0)] to the update() function. In this case, the first element of tuple is used as the key and the second element is used as the value.

We can find the length of a dictionary by using the len() function.

```
country_capitals = {"England": "London", "Italy": "Rome"}
```

```
# get dictionary's length
```

```
print(len(country_capitals)) # Output: 2
```

```
numbers = {10: "ten", 20: "twenty", 30: "thirty"}
```

```
# get dictionary's length
```

```
print(len(numbers)) # Output: 3
```

A dictionary is an ordered collection of items (starting from Python 3.7), therefore it maintains the order of its items.

```
country_capitals = {  
    "United States": "Washington D.C.",  
    "Italy": "Rome"  
}  
# print dictionary keys one by one  
for country in country_capitals:  
    print(country)  
# print dictionary values one by one  
for country in country_capitals:  
    capital = country_capitals[country]  
    print(capital)
```

### Output

```
United States  
Italy  
Washington  
D.C. Rome
```



The `keys()` method extracts the keys of the dictionary and returns the list of keys as a view object.

```
numbers = {1: 'one', 2: 'two', 3: 'three'}
```

```
# extracts the keys of the dictionary
```

```
dictionaryKeys = numbers.keys()
```

```
print(dictionaryKeys)
```

```
# Output: dict_keys([1, 2, 3])
```

The `values()` method returns a view object that displays a list of all the values in the dictionary.

```
marks = {'Physics':67, 'Maths':87}
```

```
print(marks.values())
```

```
# Output: dict_values([67, 87])
```

The `items()` method returns a view object that displays a list of dictionary's (key, value) tuple pairs.

```
marks = {'Physics':67, 'Maths':87}
```

```
print(marks.items())
```

```
# Output: dict_items([('Physics', 67), ('Maths', 87)])
```

